Iniciando com Beautiful Soup

Para que serve

O BeautifulSoup lê o HTML de uma página e **pega só os pedaços que interessam** (textos, links, autores). O pacote se chama `beautifulsoup4`, e usamos o `requests` para baixar a página.

```
pip install beautifulsoup4 requests
```

Os 3 passos de sempre

```
import requests
from bs4 import BeautifulSoup

response = requests.get("https://quotes.toscrape.com/")      # 1) baixa a página
soup = BeautifulSoup(response.text, "html.parser")           # 2) transforma o HTML em algo
navegável
titulo = soup.find("h1").text                                # 3) procura e pega o que quer
```

- `requests.get` faz o que o navegador faz ao abrir o endereço.
- `response.text` é o HTML como texto puro.
- `"html.parser"` é o leitor de HTML que já vem com o Python.

Como o HTML se parece

Uma página é feita de **tags**, e cada tag pode ter **atributos**. Dica: no navegador, botão direito no que você quer > **Inspecionar** mostra o HTML e diz qual tag e qual `class` usar.

O que é um atributo

Uma tag do HTML tem **um nome** e pode ter **informações extras sobre ela**. Essas informações extras são os **atributos**. Eles ficam dentro da tag de abertura, no formato `nome="valor"`.

Pense num documento: a tag é o tipo de objeto (um link, uma imagem), e os atributos são as **características** dele:

Tag	Atributos (as características)
Um link (`<a>`)	para onde ele leva (`href`)
Uma imagem (`<img>`)	qual arquivo mostrar (`src`) e o texto alternativo (`alt`)
Um bloco qualquer	um "apelido" para ser identificado (`class`, `id`)

O desenho, em partes

```
<a class="tag" href="/tag/vida/">vida</a>
```

Parte	Nome	Para que serve
a	**nome da tag**	Diz o tipo do elemento (a = link)

Parte	Nome	Para que serve
`class="tag"`	**atributo** `class`	"Apelido de grupo": várias tags podem ter a mesma classe
`href="/tag/vida/"`	**atributo** `href`	O endereço para onde o link leva
`vida`	**texto de dentro**	O que aparece na tela

Cada atributo tem sempre **nome** e **valor**: `href` é o nome, `"/tag/vida/"` é o valor. Uma tag pode ter vários atributos, separados por espaço.

Atributos mais comuns

Atributo	Quem usa	O que guarda
`class`	qualquer tag	Apelido de grupo (o site usa para estilizar, e nós usamos para achar)
`id`	qualquer tag	Apelido único, de um elemento só
`href`	`<a>`	Endereço do link
`src`	`<img>`	Endereço da imagem
`alt`	`<img>`	Descrição da imagem

Por que isso importa no scraping

Os atributos são a **forma de localizar** o que você quer. No site de citações:

```
<span class="text">“Frase A.”</span>
<small class="author">Ana</small>
```

Existem muitos `<span>` na página. O que diferencia os que têm citação é o atributo `class="text"`. Por isso você escreve:

```
soup.find_all("span", class_="text")
#              │         └─ "e que tenha o atributo class igual a text"
#              └─ "tags do tipo span"
```

Os atributos também são **dados que você pode querer extrair**, como o endereço de um link.

Como o BeautifulSoup entrega os atributos

Exemplo:

```
html = '<a class="tag" href="/tag/vida/" id="t1">vida</a>'
a = soup.find("a")

a.name         # 'a'                  → nome da tag
a.text         # 'vida'               → texto de dentro
a.attrs        # {'class': ['tag'], 'href': '/tag/vida/', 'id': 't1'}   → TODOS os atributos
```

O `a.attrs` é um **dicionário**: o nome do atributo é a chave, e o valor é o conteúdo. Por isso se acessa como um dicionário:

```
a["href"]              # '/tag/vida/'
a.get("href")          # '/tag/vida/'
a["title"]             # ERRO (KeyError): a tag não tem esse atributo
a.get("title")         # None (sem erro)
a.get("title", "sem título")   # 'sem título' (valor padrão)
a.has_attr("href")     # True
```
Prefira `.get(...)`, que não quebra quando o atributo não existe.

Duas pegadinhas

1. O `class` vem como lista, mesmo com um nome só:

```
a["class"]     # ['tag']      ← lista, e não o texto 'tag'
```
Isso acontece porque uma tag pode ter várias classes (`class="tag destaque"`), e o BeautifulSoup as separa. Para o primeiro valor, use `a["class"][0]`.

2. Nem toda tag tem texto de dentro. A `<img>` não tem texto, e o conteúdo dela está nos atributos:

```
img = soup.find("img")   # <img src="foto.png" alt="Uma foto">
img.text                 # ''            ← vazio
img["src"]               # 'foto.png'    ← o que interessa está aqui
```

Procurar usando qualquer atributo

```
soup.find("a", href="/tag/vida/")            # pelo href
soup.find(attrs={"id": "t1"})                # pelo id (ou qualquer atributo)
soup.find_all("span", class_="text")         # pela classe (com o underline)
```
O `class_` tem underline porque `class` é palavra reservada do Python. Os outros atributos (`href`, `id`) usam o nome normal.

Resumo em uma frase

Tag = o tipo do elemento. **Atributo** = uma característica dela, no formato `nome="valor"`. **Texto de dentro** = o que fica entre a abertura e o fechamento. No scraping, os atributos servem para **achar** a tag certa (`class_="text"`) e para **extrair** informações que não são texto (`a["href"]`).

As ferramentas principais

Quero...	Use	Devolve
A **primeira** tag que combina	`soup.find("h1")`	uma tag (ou `None`)
Todas que combinam	`soup.find_all("a")`	lista de tags (ou `[]`)
Filtrar por classe	`find_all("span", class_="text")`	lista
O texto de dentro	`tag.text`	texto
Texto sem espaços nas pontas	`tag.get_text(strip=True)`	texto
Um atributo da tag	`tag["href"]` ou `tag.get("href")`	texto

Detalhes que costumam pegar:

- `find` devolve a **tag**, e não o texto. Para o texto, use `.text`.
- Se `find` não achar nada, devolve `None`, e `None.text` dá erro.
- `find_all` sem resultado devolve lista vazia `[]`, sem erro.

`find_all` x `find`

```
soup.find_all("small", class_="author")   # [<small>Ana</small>,
<small>Bruno</small>]
soup.find("small", class_="author")       # <small>Ana</small>
```

O resultado do `find_all` é uma lista (tecnicamente um `ResultSet`, que se comporta como lista): dá para usar `len()`, `[0]` e `for`. Cada item é uma **tag**, e não um texto.

Exemplo completo e boas práticas

Cada citação do site fica num bloco `<div class="quote">` com o texto, o autor e as tags. O exemplo percorre cada bloco:

```
for bloco in soup.find_all("div", class_="quote"):          # um bloco por citação
    autor  = bloco.find("small", class_="author").text      # busca DENTRO do bloco
    texto  = bloco.find("span", class_="text").text.strip("“”")   # strip tira as
aspas
    tags   = [t.text for t in bloco.find_all("a", class_="tag")]  # lista de textos
    print(autor, "->", texto, tags)
# Ana   -> Frase A. ['vida', 'amor']
# Bruno -> Frase B. ['paz']
```

Buscar **dentro de cada bloco** (`bloco.find`) é mais seguro do que criar duas listas e juntar com `zip`: cada citação sempre fica com o seu autor. O `zip` só funciona bem quando as duas listas têm o mesmo tamanho e a mesma ordem.

Boas práticas

- Use `timeout=10` no `requests.get` para não travar se o site não responder.
- Em scraping de várias páginas, faça uma pausa (`time.sleep(1)`) entre elas.
- Use sites feitos para treino, como o quotes.toscrape.com, e leia os termos de uso de qualquer outro site.
- Se `find` pode não achar nada, confira o `None` antes de usar `.text`.